

2 — Closure: a function that keeps its variables alive

SCOPE PRIVACY STATE

Q. `createClapButton()` has returned, so why can `article.total()` still read `claps`?

ANSWER. A function's local variables normally vanish when it returns. But if the function hands back something that still references those variables, they stay alive. The bundle of a function plus the variables it captured is a closure, and the captured `claps` lives on, reachable only through `clap` and `total`.

WHY IT WORKS. Both `clap` and `total` were declared inside `createClapButton`, so they close over the *same* `claps`. When the factory returns, those two methods still hold it, so JavaScript keeps the variable in memory. Nothing outside can name `claps` directly, which makes it private state you can only touch through the methods.

examples/clapButton.js

```
function createClapButton() {  
  let claps = 0; // private,  
  starts at 0  
  return {  
    clap() { claps++; }, // increments  
    the captured claps  
    total() { return claps; } // reads the  
    captured claps  
  };  
}  
const article = createClapButton(); // the scope  
stays alive  
article.clap(); // claps → 1  
article.clap(); // claps → 2  
console.log(article.total()); // prints 2  
console.log(article.claps); // undefined –  
sealed inside
```

ONE HIDDEN VARIABLE

```
let claps = 0
```

PRIVATE · SHARED · ALIVE



```
clap()
```

WRITES CLAPS



```
total()
```

READS CLAPS

- ① **Summary.** A closure keeps a function's variables alive after it returns, shared between the methods that captured them and unreachable from anywhere else. Private state is powerful, but it is trapped inside one function — what if many files needed the same value?